

Files and directories

Files

- .NET has ready-made classes for creating, reading, writing and deleting files
- StreamWriter and StreamReader are the standard file access classes
- can map different encodings
- functions are similar to console
- Streams must be opened and also closed
 - GC does not intervene here, because opened files are considered as "unmanaged resource"

StreamWriter – System.IO

- with a StreamWriter files can be created and content can be written into them
- the Encoding property can be used to specify the encoding of the output
- with Write() and WriteLine() content can be passed to the stream

```
StreamWriter sw = new StreamWriter("Testdatei.txt");  
sw.WriteLine("Testzeile1");  
sw.WriteLine("Testzeile2");  
sw.Close();
```

StreamReader – System.IO

- with a StreamReader contents of a file can be read out
- the property EndOfStream can be used to check if the stream contains further data
- with Read() and ReadLine() contents can be read from the stream

```
StreamReader sr = new StreamReader("Testdatei.txt");  
while (!sr.EndOfStream)  
{  
    string zeile = sr.ReadLine();  
}  
sr.Close();
```

Using-Statement

- Classes that manage external resources such as files or database servers should implement the IDisposable interface.
- Dispose() clears all external resources, i.e. close files or close DB connection
- With the using statement Dispose() is called automatically as soon as the end of the using block is reached.

```
using(StreamReader reader = new StreamReader("test.txt"))  
{  
    //Read operations  
    //At the end of the block, the reader gets disposed automatically  
}
```

SaveFileDialog – System.Windows.Forms

- with a SaveFileDialog a path and a file name can be selected

- the Filter property allows to allow only certain file extensions

- the FileName property can be used to predefine a path or to retrieve the selected path

```
SaveFileDialog saveFileDialog = new SaveFileDialog();
saveFileDialog.FileName = "Personen.pl";
saveFileDialog.Filter = "Personenliste|*.pl|Textdokument|*.txt";
if (saveFileDialog.ShowDialog() == DialogResult.OK)
{
    string pfadZuDatei = saveFileDialog.FileName;
}
```

OpenFileDialog – System.Windows.Forms

- same functionality as SaveFileDialog
- the file path must exist to confirm the dialog

```
OpenFileDialog openFileDialog = new OpenFileDialog();
openFileDialog.FileName = "Personen.pl";
openFileDialog.Filter = "Personenliste|*.pl|Textdokument|*.txt";
if (openFileDialog.ShowDialog() == DialogResult.OK)
{
    string pfadZuDatei = openFileDialog.FileName;
}
```

JSON – Newtonsoft.Json (NuGet-Packet)

- an object can be written with the current state e.g. into a file and later be read in again
- the class JsonConvert takes care of the conversion of the contents
- with the method SerializeObject of the class JsonConvert an arbitrary object can be converted into a JSON string (serialized)
- with the method DeserializeObject with type <T> of the class JsonConvert an arbitrary object can be converted into a JSON string (serialized)

```
List<string> strings = new List<string> { "Wert1", "Wert2", "Wert3" };
```

```
string jsonString = JsonConvert.SerializeObject(strings);
```

```
List<string> stringListe = JsonConvert.DeserializeObject<List<string>>(jsonString);
```


JSON – System.Text.Json (from .NET Core 3.0)

- new JSON converter included directly in .NET Core
- less function than Newtonsoft.Json
- a referenced loop leads to an error message
- much faster than Newtonsoft.Json
- but is type dependent (int => int only) case sensitive, property names only with double quotes

```
List<string> strings = new List<string> { "Wert1", "Wert2", "Wert3" };
```

```
string jsonString = JsonSerializer.Serialize(strings);
```

```
List<string> stringListe = JsonSerializer.Deserialize<List<string>>(jsonString);
```

XML – System.Xml.Serialization

- with an XmlSerializer an object can be written directly into a file using a TextWriter (e.g. StreamWriter)
- in contrast to the JSON classes a separate object of the class XmlSerializer must be created for XML, which must have an instance of the type of the object to be serialized in the constructor
- the TextWriter as well as the object to be serialized is passed via the method Serialize of the XmlSerializer object

```
List<string> liste = new List<string> { "Wert1", "Wert2", "Wert3" };  
  
XmlSerializer serializer = new XmlSerializer(typeof(List<string>));  
  
StreamWriter sw = new StreamWriter("liste.xml");  
serializer.Serialize(sw, liste);  
sw.Close();
```